referenced here. If this image is already available in `public/assets` as a fingerprinted file, then that path is referenced.
If you want to use a data URI - a method of embedding the image data directly into the CSS file - you can use the `asset_data_uri` helper.

```
#logo { background: url(<%= asset_data_uri 'logo.png' %>) }
```

This inserts a correctly-formatted data URI into the CSS source.

Note that the closing tag cannot be of the style `-%>`.

2.3.2 CSS and Sass

When using the asset pipeline, paths to assets must be re-written and `sass-rails` provides `-url`and `-path` helpers (hyphenated in Sass, underscored in Ruby) for the following asset classes: image, font, video, audio, JavaScript and stylesheet.

- `image-url("rails.png")` becomes `url(/assets/rails.png)`
- `image-path("rails.png")` becomes `"/assets/rails.png"`.

The more generic form can also be used:

- `asset-url("rails.png")` becomes `url(/assets/rails.png)`
- `asset-path("rails.png")` becomes `"/assets/rails.png"`

2.3.3 JavaScript/CoffeeScript and ERB

If you add an `erb` extension to a JavaScript asset, making it something such as `application.js.erb`, you can then use the `asset_path` helper in your JavaScript code:

```
$('#logo').attr({ src: "<%= asset_path('logo.png') %>" });
```

This writes the path to the particular asset being referenced.

Similarly, you can use the `asset_path` helper in CoffeeScript files with `erb` extension (e.g., `application.coffee.erb`):

```
$('#logo').attr src: "<%= asset_path('logo.png') %>"
```

2.4 Manifest Files and Directives

Sprockets uses manifest files to determine which assets to include and serve. These manifest files contain *directives* - instructions that tell Sprockets which files to require in order to build a single CSS or JavaScript file. With these directives, Sprockets loads the files specified, processes them if necessary, concatenates them into one single file and then compresses them (if `Rails.application.config.assets.compress` is true). By serving one file rather than many, the load time of pages can be greatly reduced because the browser makes fewer requests. Compression also reduces file size, enabling the browser to download them faster.
For example, a new Rails 4 application includes a default `app/assets/javascripts/application.js` file containing the following lines:

```
// ...
//= require jquery
//= require jquery_ujs
//= require_tree .
```

In JavaScript files, Sprockets directives begin with `//=`. In the above case, the file is using the `require` and the `require_tree` directives. The `require` directive is used to tell Sprockets the files you wish to require. Here, you are requiring the files `jquery.js` and `jquery_ujs.js` that are available somewhere in the search path for Sprockets. You need not supply the extensions explicitly. Sprockets assumes you are requiring a `.js` file when done from within a `.js` file.